Development Report: Prime Number Checker

Date: April 21, 2026
Project: Integer Analysis Module (Prime Checker)
Language: Python 3.x

1. Program Description

The program is designed to determine the mathematical status of an integer. It classifies the user-inputted number into three distinct categories:

- **Prime number:** Numbers with exactly two divisors.
- **Composite number:** Numbers with more than two divisors.
- **Neither prime nor composite:** Specific case for numbers less than 2.

2. Algorithm and Optimization

The core logic uses an optimized trial division method. The search for divisors is limited to the range **[2, √n]**. This mathematical optimization ensures that the program can handle large inputs efficiently, as any composite number must have at least one factor less than or equal to its square root.

3. Source Code

```
# Creating a function that checks whether a number is prime or composite
def isPrime(n):
    if n < 2:
        return "Neither prime nor composite"
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return "Composite number"
    return "Prime number"

# Testing function
try:
    a = int(input("Input integer number: "))
    print(f"{a} - {isPrime(a)}")
# If a non-integer number is entered, an error is displayed
except ValueError:
```

```
    print("Please, enter integer number!!!")
```

4. Test Results

Input Data	Expected Result	Actual Output	Status
12.3	ValueError Exception	"Please, enter integer number!!!"	**PASSED**
1	Neither prime nor composite	"1 - Neither prime nor composite"	**PASSED**
7	Prime number	"7 - Prime number"	**PASSED**
14	Composite number	"14 - Composite number"	**PASSED**

5. Error Handling

The implementation includes a `try-except` block to handle `ValueError`. This ensures the application remains stable when users provide invalid inputs such as floating-point numbers or alphabetical characters, providing clear feedback instead of a system crash.

6. Conclusion

The algorithm is mathematically sound, performance-optimized, and robust against user error. It is ready for deployment or further integration into larger computational modules.